

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Computer Vision and Image Processing (CSDS119)
Lab Assignment 2

Problem Statement:**Code:**

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 typedef struct {
7     char magic[3];
8     int w, h, max_val;
9     int channels;
10    int bytes_per_sample;
11    int bytes_per_pixel;
12    long long total_pixels;
13    long long expected_bytes;
14    long header_offset;
15    long file_size;
16    unsigned char *data;
17 } ImageHeader;
18
19 static void skip_comments(FILE *fp) {
20     int ch;
21
22     while ((ch = fgetc(fp)) != EOF) {
23         if (isspace(ch))
24             continue;
25
26         if (ch == '#') {
27             while ((ch = fgetc(fp)) != EOF &&
28                 ch != '\n');
29             continue;
30         }
31
32         ungetc(ch, fp);
33         break;
34     }
35 }
36
37 ImageHeader* read_image_header_and_data(
38     const char *filename
39 ) {
40     FILE *fp =
41         fopen(filename, "rb");
```

```
42
43     if (!fp) {
44         fprintf(
45             stderr,
46             "Error: Cannot open file '%s'\n",
47             filename
48         );
49         return NULL;
50     }
51
52     fseek(fp, 0, SEEK_END);
53     long file_size =
54         ftell(fp);
55
56     fseek(fp, 0, SEEK_SET);
57
58     char magic[3] = {0};
59
60     if (fscanf(fp, "%2s", magic) != 1) {
61         fprintf(
62             stderr,
63             "Error: Invalid header in '%s'\n",
64             filename
65         );
66
67         fclose(fp);
68         return NULL;
69     }
70
71     if (strcmp(magic, "P5") != 0 &&
72         strcmp(magic, "P6") != 0) {
73
74         fprintf(
75             stderr,
76             "Error: Unsupported magic '%s'. "
77             "Only P5 and P6 supported.\n",
78             magic
79         );
80
81         fclose(fp);
82         return NULL;
83     }
84
85     skip_comments(fp);
86
87     int w = 0;
88     int h = 0;
89     int max_val = 0;
90
91     if (fscanf(fp, "%d", &w) != 1) {
```

```
92     fclose(fp);
93     return NULL;
94 }
95
96 skip_comments(fp);
97
98 if (fscanf(fp, "%d", &h) != 1) {
99     fclose(fp);
100    return NULL;
101 }
102
103 skip_comments(fp);
104
105 if (fscanf(fp, "%d", &max_val) != 1) {
106     fclose(fp);
107     return NULL;
108 }
109
110 fgetc(fp);
111
112 long header_offset =
113     ftell(fp);
114
115 ImageHeader *img =
116     (ImageHeader *)calloc(
117         1,
118         sizeof(ImageHeader)
119     );
120
121 if (!img) {
122     fclose(fp);
123     return NULL;
124 }
125
126 strcpy(img->magic, magic);
127
128 img->w = w;
129 img->h = h;
130 img->max_val = max_val;
131
132 img->channels =
133     (strcmp(magic, "P6") == 0)
134     ? 3
135     : 1;
136
137 img->bytes_per_sample =
138     (max_val <= 255)
139     ? 1
140     : 2;
```

```
142     img->bytes_per_pixel =
143         img->channels *
144         img->bytes_per_sample;
145
146     img->total_pixels =
147         (long long)w * h;
148
149     img->expected_bytes =
150         img->total_pixels *
151         img->bytes_per_pixel;
152
153     img->header_offset =
154         header_offset;
155
156     img->file_size =
157         file_size;
158
159     img->data =
160         (unsigned char *)malloc(
161             img->expected_bytes
162         );
163
164     if (img->data) {
165         size_t read_bytes =
166             fread(
167                 img->data,
168                 1,
169                 img->expected_bytes,
170                 fp
171             );
172
173         if (read_bytes !=
174             (size_t)img->expected_bytes) {
175
176             fprintf(
177                 stderr,
178                 "Warning: Expected %lld bytes, "
179                 "read %zu in '%s'\n",
180                 img->expected_bytes,
181                 read_bytes,
182                 filename
183             );
184         }
185     }
186
187     fclose(fp);
188     return img;
189 }
190
191 void free_image_header(
```

```
192     ImageHeader *img
193 ) {
194     if (img) {
195         if (img->data)
196             free(img->data);
197
198         free(img);
199     }
200 }
201
202 void analyze_image_file(
203     const char *filename
204 ) {
205     printf(
206         "\n
=====
n"
207     );
208
209     printf(
210         " HEADER & DATA ANALYSIS: %s\n",
211         filename
212     );
213
214     printf(
215         "
=====
n"
216     );
217
218     ImageHeader *img =
219         read_image_header_and_data(
220             filename
221         );
222
223     if (!img)
224         return;
225
226     printf(
227         "1. Image Format & Magic Number:\n"
228     );
229
230     printf(
231         "    Magic Number    : %s (%s)\n",
232         img->magic,
233         (strcmp(img->magic, "P5") == 0)
234         ? "Binary Grayscale (PGM)"
235         : "Binary Truecolor RGB (PPM)"
236     );
237
```

```
238 printf(  
239     "    Architectural Meaning : "  
240     "Specifies file structure, raster layout, "  
241     "and color channels.\n"  
242 );  
243  
244 printf(  
245     "\n2. Header Fields:\n"  
246 );  
247  
248 printf(  
249     "    Width           : %d pixels\n",  
250     img->w  
251 );  
252  
253 printf(  
254     "    Height          : %d pixels\n",  
255     img->h  
256 );  
257  
258 printf(  
259     "    MaxVal           : %d (%s-bit depth)\n",  
260     img->max_val,  
261     (img->max_val <= 255)  
262     ? "8"  
263     : "16"  
264 );  
265  
266 printf(  
267     "\n3. Pixel & Channel Calculations:\n"  
268 );  
269  
270 printf(  
271     "    Total Pixels      : %lld "  
272     "(Width x Height = %d x %d)\n",  
273     img->total_pixels,  
274     img->w,  
275     img->h  
276 );  
277  
278 printf(  
279     "    Color Channels    : %d (%s)\n",  
280     img->channels,  
281     (img->channels == 1)  
282     ? "Grayscale"  
283     : "RGB"  
284 );  
285  
286 printf(  
287     "    Bytes per Sample: %d byte(s) "
```

```
288     "[MaxVal %s 255]\n",
289     img->bytes_per_sample,
290     (img->max_val <= 255)
291     ? "<="
292     : ">"
293 );
294
295 printf(
296     "    Bytes per Pixel : %d byte(s) "
297     "(Channels x Bytes/Sample = %d x %d)\n",
298     img->bytes_per_pixel,
299     img->channels,
300     img->bytes_per_sample
301 );
302
303 printf(
304     "\n4. Expected Image Data Size:\n"
305 );
306
307 printf(
308     "    Formula          : "
309     "Width x Height x Channels x BytesPerSample\n"
310 );
311
312 printf(
313     "    Calculation      : "
314     "%d x %d x %d x %d = %lld bytes\n",
315     img->w,
316     img->h,
317     img->channels,
318     img->bytes_per_sample,
319     img->expected_bytes
320 );
321
322 long actual_data_in_file =
323     img->file_size -
324     img->header_offset;
325
326 printf(
327     "\n5. File Integrity Verification:\n"
328 );
329
330 printf(
331     "    Header Size      : %ld bytes "
332     "(raster payload offset)\n",
333     img->header_offset
334 );
335
336 printf(
337     "    Total File Size : %ld bytes\n",
```

```
338     img->file_size
339 );
340
341 printf(
342     "    Available Data   : %ld bytes "
343     "(Total File Size - Header Size)\n",
344     actual_data_in_file
345 );
346
347 if (actual_data_in_file ==
348     img->expected_bytes) {
349
350     printf(
351         "    Status           : [VERIFIED] "
352         "Expected data matches actual file "
353         "payload exactly.\n"
354     );
355 }
356 else {
357     printf(
358         "    Status           : [MISMATCH] "
359         "File may be truncated, corrupt, "
360         "or misparsed!\n"
361     );
362 }
363
364 printf(
365     "\n6. First 10 Pixel Values:\n    "
366 );
367
368 for (int i = 0;
369     i < 10 &&
370     i < img->total_pixels;
371     i++) {
372
373     if (img->channels == 1) {
374
375         if (img->bytes_per_sample == 1) {
376
377             printf(
378                 "[%d] ",
379                 img->data[i]
380             );
381         }
382         else {
383
384             int val =
385                 (img->data[i * 2] << 8) |
386                 img->data[i * 2 + 1];
387
```

```
388         printf(
389             "[%d] ",
390             val
391         );
392     }
393 }
394 else {
395
396     if (img->bytes_per_sample == 1) {
397
398         printf(
399             "(%d,%d,%d) ",
400             img->data[i * 3],
401             img->data[i * 3 + 1],
402             img->data[i * 3 + 2]
403         );
404     }
405     else {
406
407         int r =
408             (img->data[i * 6] << 8) |
409             img->data[i * 6 + 1];
410
411         int g =
412             (img->data[i * 6 + 2] << 8) |
413             img->data[i * 6 + 3];
414
415         int b =
416             (img->data[i * 6 + 4] << 8) |
417             img->data[i * 6 + 5];
418
419         printf(
420             "(%d,%d,%d) ",
421             r,
422             g,
423             b
424         );
425     }
426 }
427 }
428
429 printf("\n");
430
431 if (img->bytes_per_sample == 2) {
432
433     printf(
434         "    16-Bit Interpretation: "
435         "2 consecutive bytes per sample "
436         "in Big-Endian order: "
437         "(MSB << 8) | LSB.\n"
```

```
438     );
439 }
440
441 free_image_header(img);
442 }
443
444 void demonstrate_extension_vs_format(
445     void
446 ) {
447     printf(
448         "\n
=====
n"
449     );
450
451     printf(
452         " EXTENSION VERSUS FORMAT EVALUATION\n"
453     );
454
455     printf(
456         "
=====
n"
457     );
458
459     printf(
460         "    Finding: File extensions "
461         " (.pgm, .ppm, .dat) are merely OS naming hints.\n"
462     );
463
464     printf(
465         "    Authoritative Source: "
466         "The magic number in the file header "
467         "(P5 vs P6)\n"
468     );
469
470     printf(
471         "    is the ONLY reliable identifier "
472         "that dictates binary decoding rules.\n"
473     );
474
475     printf(
476         "    Proof: Renaming 'sample.ppm' to "
477         "'sample.pgm' does not change the pixel\n"
478     );
479
480     printf(
481         "    interleaving (RGB triplets). "
482         "The header magic number ensures correct parsing.\n"
483     );
```

```
484 }
485
486 void critical_thinking_analysis(
487     void
488 ) {
489     printf(
490         "\n
=====
n"
491     );
492
493     printf(
494         " CRITICAL-THINKING ANALYSIS "
495         "(Dimensions: 640 x 480 = 307,200 pixels)\n"
496     );
497
498     printf(
499         "
=====
n"
500     );
501
502     printf(
503         " Configuration | "
504         "Formula (W x H x C x B) | Expected Data Size\n"
505     );
506
507     printf(
508         " -----+
509         "-----+-----\n"
510     );
511
512     printf(
513         " (i) PGM (MaxVal = 255, 8-bit, 1 ch) | "
514         "640 x 480 x 1 x 1 | 307,200 bytes\n"
515     );
516
517     printf(
518         " (ii) PPM (MaxVal = 255, 8-bit, 3 ch) | "
519         "640 x 480 x 3 x 1 | 921,600 bytes\n"
520     );
521
522     printf(
523         " (iii) PGM (MaxVal = 65535, 16-bit, 1 ch) | "
524         "640 x 480 x 1 x 2 | 614,400 bytes\n"
525     );
526
527     printf(
528         " (iv) PPM (MaxVal = 65535, 16-bit, 3 ch) | "
529         "640 x 480 x 3 x 2 | 1,843,200 bytes\n"
```

```
530     );
531
532     printf(
533         "
534         n"
535     );
536     printf(
537         "    Scientific Explanation:\n"
538     );
539
540     printf(
541         "    -> Spatial dimensions (640x480) define "
542         "only the spatial sampling grid (pixel count).\n"
543     );
544
545     printf(
546         "    -> Total byte size scales linearly "
547         "with two orthogonal dimensions:\n"
548     );
549
550     printf(
551         "        1. Color Channels (C): "
552         "Grayscale (1) vs RGB Color (3) -> 3x multiplier.\n"
553     );
554
555     printf(
556         "        2. Quantization Bit Depth (B): "
557         "8-bit (1 byte) vs 16-bit (2 bytes) -> 2x multiplier.\n"
558     );
559
560     printf(
561         "
562         =====\n\n"
563     );
564 }
565
566 int main(
567     int argc,
568     char *argv[]
569 ) {
570     const char *pgm_file =
571         (argc > 1)
572         ? argv[1]
573         : "sample_01.pgm";
574
575     const char *ppm_file =
```

```
576     (argc > 2)
577     ? argv[2]
578     : "sample_01.ppm";
579
580     analyze_image_file(
581         pgm_file
582     );
583
584     analyze_image_file(
585         ppm_file
586     );
587
588     demonstrate_extension_vs_format();
589
590     critical_thinking_analysis();
591
592     return 0;
593 }
```

Output:

```
=====
HEADER & DATA ANALYSIS: sample_01.pgm
=====

1. Image Format & Magic Number:
   Magic Number      : P5 (Binary Grayscale (PGM))
   Architectural Meaning : Specifies file structure, raster layout,
   and color channels.

2. Header Fields:
   Width             : 640 pixels
   Height            : 426 pixels
   MaxVal            : 255 (8-bit depth)

3. Pixel & Channel Calculations:
   Total Pixels      : 272640 (Width x Height = 640 x 426)
   Color Channels    : 1 (Grayscale)
   Bytes per Sample: 1 byte(s) [MaxVal <= 255]
   Bytes per Pixel  : 1 byte(s) (Channels x Bytes/Sample = 1 x 1)

4. Expected Image Data Size:
   Formula           : Width x Height x Channels x BytesPerSample
   Calculation       : 640 x 426 x 1 x 1 = 272640 bytes

5. File Integrity Verification:
   Header Size       : 15 bytes (raster payload offset)
   Total File Size   : 272655 bytes
   Available Data    : 272640 bytes (Total File Size - Header Size)
   Status            : [VERIFIED] Expected data matches actual file
   payload exactly.
```

6. First 10 Pixel Values:

[141] [136] [133] [133] [134] [130] [125] [122] [123] [123]

=====

HEADER & DATA ANALYSIS: sample_01.ppm

1. Image Format & Magic Number:

Magic Number : P6 (Binary Truecolor RGB (PPM))

Architectural Meaning : Specifies file structure, raster layout, and color channels.

2. Header Fields:

Width : 640 pixels

Height : 426 pixels

MaxVal : 255 (8-bit depth)

3. Pixel & Channel Calculations:

Total Pixels : 272640 (Width x Height = 640 x 426)

Color Channels : 3 (RGB)

Bytes per Sample: 1 byte(s) [MaxVal <= 255]

Bytes per Pixel : 3 byte(s) (Channels x Bytes/Sample = 3 x 1)

4. Expected Image Data Size:

Formula : Width x Height x Channels x BytesPerSample

Calculation : 640 x 426 x 3 x 1 = 817920 bytes

5. File Integrity Verification:

Header Size : 56 bytes (raster payload offset)

Total File Size : 817976 bytes

Available Data : 817920 bytes (Total File Size - Header Size)

Status : [VERIFIED] Expected data matches actual file payload exactly.

6. First 10 Pixel Values:

(141,141,141) (136,136,136) (133,133,133) (133,133,133)

(134,134,134) (130,130,130) (125,125,125) (122,122,122)

(123,123,123) (123,123,123)

=====

EXTENSION VERSUS FORMAT EVALUATION

Finding: File extensions (.pgm, .ppm, .dat) are merely OS naming hints.

Authoritative Source: The magic number in the file header (P5 vs P6) is the ONLY reliable identifier that dictates binary decoding rules

Proof: Renaming 'sample.ppm' to 'sample.pgm' does not change the pixel interleaving (RGB triplets). The header magic number ensures

correct parsing.

=====

CRITICAL-THINKING ANALYSIS (Dimensions: 640 x 480 = 307,200 pixels)

=====

Configuration	Formula (W x H x C x B)
Expected Data Size	

(i) PGM (MaxVal = 255, 8-bit, 1 ch)	640 x 480 x 1 x 1
307,200 bytes	
(ii) PPM (MaxVal = 255, 8-bit, 3 ch)	640 x 480 x 3 x 1
921,600 bytes	
(iii) PGM (MaxVal = 65535, 16-bit, 1 ch)	640 x 480 x 1 x 2
614,400 bytes	
(iv) PPM (MaxVal = 65535, 16-bit, 3 ch)	640 x 480 x 3 x 2
1,843,200 bytes	

Scientific Explanation:

- > Spatial dimensions (640x480) define only the spatial sampling grid (pixel count).
- > Total byte size scales linearly with two orthogonal dimensions:
 1. Color Channels (C): Grayscale (1) vs RGB Color (3) -> 3x multiplier.
 2. Quantization Bit Depth (B): 8-bit (1 byte) vs 16-bit (2 bytes) -> 2x multiplier.

Analysis:

1. Identification of Image Format and Magic Number

The program successfully reads both binary image files, `sample_01.pgm` and `sample_01.ppm`, in binary mode. The PGM file begins with the magic number P5, while the PPM file begins with P6. The magic number is the authoritative identifier of the Netpbm file format and determines how the binary raster data should be interpreted. This directly satisfies the requirement of identifying the image format from the file header rather than relying on the file extension. The assignment specifies that P5 represents binary PGM and P6 represents binary PPM. :contentReference[oaicite:0]index=0

Table 1: Image Format Identification

Parameter	sample_01.pgm	sample_01.ppm
Magic Number	P5	P6
Format	Binary PGM	Binary PPM
Image Type	Grayscale	Truecolor RGB
Number of Channels	1	3

The PGM image therefore contains one intensity sample per pixel, whereas the PPM image contains three samples per pixel corresponding to red, green, and blue.

2. Reading and Analysis of Header Fields

The program extracts the width, height, and MaxVal directly from the image headers. It also implements a header parser capable of skipping whitespace and comments beginning with #. For both test images, the width is 640 pixels, the height is 426 pixels, and the maximum intensity value is 255. Therefore, both images use an 8-bit representation.

Table 2: Header Fields of the Test Images

Header Field	PGM	PPM
Width	640 pixels	640 pixels
Height	426 pixels	426 pixels
MaxVal	255	255
Bit Depth	8-bit	8-bit

The header information is sufficient to determine the basic structure and storage requirements of the image data without using an image-processing library.

3. Determination of Total Number of Pixels

The total number of pixels is calculated using

$$\text{Total Pixels} = \text{Width} \times \text{Height}.$$

For both images,

$$640 \times 426 = 272640.$$

Thus, both the PGM and PPM images contain 272640 spatial pixel locations. Although the number of spatial pixels is the same, the amount of image data is different because a PPM pixel contains three colour samples.

Table 3: Pixel Count Calculation

Parameter	PGM	PPM
Width	640	640
Height	426	426
Total Pixels	640×426	640×426
Total Pixel Locations	272640	272640

4. Determination of Bytes per Sample and Bytes per Pixel

Since the MaxVal of both images is 255, each sample requires one byte. For PGM, one pixel contains one grayscale sample, resulting in one byte per pixel. For PPM, each pixel contains three samples, namely R, G, and B. Therefore, each PPM pixel requires three bytes.

Table 4: Bytes per Sample and Pixel

Parameter	PGM (P5)	PPM (P6)
MaxVal	255	255
Bytes per Sample	1	1
Colour Channels	1	3
Bytes per Pixel	1	3

The result demonstrates that the number of colour channels directly affects the storage requirement of each pixel.

5. Expected Image-Data Size

The expected image-data size is calculated using

$$\text{Expected Data Size} = \text{Width} \times \text{Height} \times \text{Channels} \times \text{Bytes per Sample}.$$

For the PGM image,

$$640 \times 426 \times 1 \times 1 = 272640 \text{ bytes}.$$

For the PPM image,

$$640 \times 426 \times 3 \times 1 = 817920 \text{ bytes}.$$

Table 5: Expected Image-Data Size

Parameter	PGM	PPM
Width	640	640
Height	426	426
Channels	1	3
Bytes per Sample	1	1
Calculation	$640 \times 426 \times 1 \times 1$	$640 \times 426 \times 3 \times 1$
Expected Data Size	272640 bytes	817920 bytes

Although both files have identical spatial dimensions, the PPM file requires three times as much pixel storage because every pixel contains three colour components.

6. Reading and Verification of Image Data

After processing the header, the program dynamically allocates memory for the calculated number of bytes and uses `fread()` to read the image payload. It does not assume a fixed image size. The actual amount of data available in the file is calculated using the total file size and the position at which the raster data begins.

Table 6: File Integrity Verification

Parameter	sample_01.pgm	sample_01.ppm
Header/Payload Offset	15 bytes	56 bytes
Total File Size	272655 bytes	817976 bytes
Available Pixel Data	272640 bytes	817920 bytes
Expected Pixel Data	272640 bytes	817920 bytes
Verification Status	VERIFIED	VERIFIED

For both files, the available payload exactly matches the expected image-data size. Therefore, the files are complete and correctly interpreted by the program. No evidence of truncation, corruption, or incorrect parsing was observed.

7. Analysis of the First Ten Pixels

The program displays the first ten pixel values after reading the binary payload. For the PGM image, each value represents one grayscale intensity. For the PPM image, each pixel is displayed as an RGB triplet.

Table 7: First Ten Pixel Values

Pixel	PGM Value	PPM RGB Value
1	141	(141, 141, 141)
2	136	(136, 136, 136)
3	133	(133, 133, 133)
4	133	(133, 133, 133)
5	134	(134, 134, 134)
6	130	(130, 130, 130)
7	125	(125, 125, 125)
8	122	(122, 122, 122)
9	123	(123, 123, 123)
10	123	(123, 123, 123)

The RGB components of the first ten PPM pixels are identical, for example (141, 141, 141) and (136, 136, 136). This indicates that these particular pixels represent grayscale colours stored in the RGB image.

For a 16-bit image, each sample occupies two consecutive bytes. The program interprets these bytes in big-endian order using

$$\text{Value} = (\text{MSB} \ll 8) \mid \text{LSB}.$$

Thus, the implementation is capable of determining the value of samples when MaxVal is greater than 255, as required by the assignment. :contentReference[oaicite:1]index=1

8. Extension Versus Actual File Format

The program demonstrates that the file extension is not sufficient to identify the actual image format. File extensions such as .pgm, .ppm, or even .dat are only naming conventions. The actual format is determined by the magic number stored inside the file.

Table 8: Extension Versus Format

Aspect	PGM	PPM
Typical Extension	.pgm	.ppm
Magic Number	P5	P6
Channels	1	3
Binary Pixel Structure	Grayscale samples	RGB triplets
Reliable Format Identifier	Header Magic Number	Header Magic Number

For example, renaming a PPM file from `sample.ppm` to `sample.pgm` does not change its internal P6 magic number or its RGB pixel organization. Therefore, the program must inspect the header rather than relying on the filename extension.

9. Critical-Thinking Analysis

The assignment considers an image of dimensions 640×480 with 307200 pixels and asks for the expected data size under different combinations of image format and bit depth. :contentReference[oaicite:2]index=2

Table 9: Critical-Thinking Storage Analysis for a 640×480 Image

Configuration	Calculation	Expected Size
PGM, 8-bit, 1 channel	$640 \times 480 \times 1 \times 1$	307200 bytes
PPM, 8-bit, 3 channels	$640 \times 480 \times 3 \times 1$	921600 bytes
PGM, 16-bit, 1 channel	$640 \times 480 \times 1 \times 2$	614400 bytes
PPM, 16-bit, 3 channels	$640 \times 480 \times 3 \times 2$	1843200 bytes

The results demonstrate that the spatial dimensions determine the number of pixel locations, but they do not completely determine the amount of storage required. The storage requirement also depends on the number of colour channels and the number of bytes required for each sample.

For an 8-bit grayscale image, one pixel requires one byte. Changing from PGM to PPM introduces three colour channels and therefore increases the storage requirement by a factor of three. Similarly, increasing the bit depth from 8-bit to 16-bit changes the storage per sample from one byte to two bytes, producing a factor of two increase.

Table 10: Effect of Channels and Bit Depth on Storage

Change	Multiplier	Reason
PGM $\rightarrow$ PPM	$3\times$	1 channel $\rightarrow$ 3 channels
8-bit $\rightarrow$ 16-bit	$2\times$	1 byte/sample $\rightarrow$ 2 bytes/sample
PGM 8-bit $\rightarrow$ PPM 16-bit	$6\times$	$3 \text{ channels} \times 2 \text{ bytes/sample}$

Therefore, two images can have exactly the same width and height but require significantly different amounts of storage because their colour representation and quantization bit depth are different.

10. Overall Conclusion

The program successfully performs the required PGM/PPM header analysis without using an image-processing library. It identifies the format using the magic number, extracts the width, height, and MaxVal, calculates the total number of pixels, determines the number of channels and bytes per sample, and computes the expected image-data size.

The experimental results show that `sample_01.pgm` is a P5 binary grayscale image containing 272640 pixels and requiring 272640 bytes of pixel data. `sample_01.ppm` is a P6 binary RGB image containing the same 272640 spatial pixels but requiring 817920 bytes because each pixel contains three colour samples.

Both files passed the integrity verification because their expected data sizes exactly matched the available payload sizes. The first ten pixels of the PPM image contain equal R, G, and B values, indicating grayscale-valued pixels represented within an RGB format.

The critical-thinking analysis further confirms that image dimensions alone do not determine file size. Colour channels and bit depth directly affect the number of bytes required to represent each pixel. Thus, correct interpretation of the image header is essential for reading the appropriate amount of binary image data and for avoiding incorrect or corrupted image interpretation.